

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ ПО ВЫПОЛНЕНИЮ СКВОЗНОГО ПРОЕКТА

ВЕХА 4. ИНФРАСТРУКТУРА, ДЕПЛОЙ И ПРОДУКТИВНЫЙ МОНИТОРИНГ

Проект выполняется **КОМАНДОЙ**, в которой каждый участник имеет свою **РОЛЬ**. На этапе Вехи 4 общая командная цель - превратить лучшую математическую модель в изолированный, отчуждаемый промышленный программный сервис, оснащенный контуром автоматического мониторинга и переобучения.

- Роль MLOps — Координатор Вехи 4: Отвечает за архитектуру REST API, написание Dockerfile, изоляцию среды, логирование артефактов и автоматизацию пайплайнов переобучения.
- Роль ML Engineer: Осуществляет экспорт весов лучшей модели (в формат .pt, .pkl или ONNX), адаптирует скрипты инференса для работы внутри контейнера без зависимости от обучающей выборки.
- Роль Data Analyst: Проектирует логику работы скрипта-монитора, который в реальном времени рассчитывает отклонения поступающих данных и подает сигнал системе автоматизации на перезапуск обучения.

1. ВХОДНОЙ ОРГАНИЗАЦИОННЫЙ РЕГЛАМЕНТ

Перед выводом модели в продуктив команда выполняет три условия:

1. Замораживание моделей: на основе результатов Вех 2 и 3 команда выбирает **одну эталонную модель** (победителя по метрикам) и сериализует её веса в отчуждаемый файл.
2. Архитектурное планирование: MLOps-инженер составляет схему контрактов API: жестко фиксирует типы входящих JSON-запросов и структуру исходящих прогнозных ответов сервиса.
3. Инфраструктурный аудит: команда проверяет работоспособность Docker-демона на рабочих машинах для сборки контейнеров.

2. ЦЕЛЬ И ЗАДАЧИ ВЕХИ 4

Цель работы - проектирование и программная реализация промышленного инференс-сервиса на базе фреймворка FastAPI, его контейнеризация с помощью Docker, интеграция систем логирования экспериментов (MLflow) и автоматизация контура продуктивного мониторинга и ретренинга моделей при деградации качества.

Задачи вехи:

- Разработать программный интерфейс REST API (на FastAPI), принимающий текущую историю ряда и возвращающий вектор прогнозов в формате JSON.
- Интегрировать инструмент MLflow (или аналог) для централизованного логирования параметров, метрик и версий весов моделей.
- Написать оптимизированный Dockerfile, собрать легковесный Docker-образ и запустить инференс-сервис в изолированном контейнере.
- Реализовать скрипт-монитор, симулирующий подачу новых продуктовых данных и вычисляющий дрейф концептов.
- Автоматизировать пайплайн ретренинга: написать сценарий (DAG для Airflow / Prefect или автономный python-триггер), который автоматически переобучает модель при получении сигнала от монитора.

3. ПОШАГОВЫЙ ХОД ВЫПОЛНЕНИЯ РАБОТЫ ДЛЯ СТУДЕНТОВ

Этап 1. Разработка REST API и логирование в MLflow

- **Действия студентов:** создать скрипт `app.py` на базе FastAPI. Описать строгую валидацию входящих данных через библиотеки `pydantic`. Настроить эндпоинт `/predict` (принимает JSON с признаками — выдает JSON с прогнозом на заданный горизонт). Интегрировать в код обучения из прошлых веж логирование: при каждом запуске скрипта в систему MLflow должны автоматически записываться: глубина окон, гиперпараметры, итоговый RMSE и сам файл с весами модели (artifacts).

- **Зона ответственности ролей:** MLOps кодирует каркас API; ML Engineer интегрирует вызовы функций предсказания модели внутрь роутов FastAPI.

Этап 2. Контейнеризация и Docker

- **Действия студентов:** в корне проекта написать текстовый файл `Dockerfile`. В качестве базового образа выбрать официальный легковесный `python:3.10-slim`. Прописать копирование кода предобработки, весов моделей и файла зависимостей (`requirements.txt`). Настроить команду запуска веб-сервера `uvicorn`. Собрать образ командой `docker build` и запустить изолированный контейнер. Проверить работоспособность сервиса через автоматическую интерактивную документацию Swagger UI (`/docs`).

- **Зона ответственности ролей:** Полностью выполняется MLOps-инженером команды.

Этап 3. Симуляция продуктива, мониторинг и ретренинг

- **Действия студентов:** написать скрипт-симулятор, который имитирует работу сервиса в реальном времени (поочередно посылает в контейнер новые порции данных из тестовой выборки Вехи 3, где заложен дрейф). Скрипт мониторинга непрерывно фиксирует истинные значения и рассчитывает ошибку. Как только ошибка превышает критический порог (произошел дрейф), скрипт автоматически посылает сигнал (триггер) на запуск пайплайна автоматического переобучения (`retrain_pipeline.py`). Модель переобучается на новых данных, её веса обновляются в MLflow, а контейнер подхватывает новую версию без остановки сервиса.

- **Зона ответственности ролей:** Data Analyst рассчитывает пороговые значения для триггера; MLOps настраивает автоматизацию перезапуска скриптов обучения в контуре ретренинга.

4. РЕГЛАМЕНТ ИСПОЛЬЗОВАНИЯ ГЕНЕРАТИВНОГО ИИ (LLM-КАНОН)

РАЗРЕШЕНО: Использовать LLM для генерации синтаксиса стандартных команд `Dockerfile` (инструкции `EXPOSE`, `WORKDIR`, `RUN pip install`), написания базовых декораторов роутинга в FastAPI (`@app.post("/predict")`) и создания шаблонов конфигурации MLflow сервера.

ЗАПРЕЩЕНО: Делегировать ИИ проектирование архитектуры взаимодействия компонентов системы и логики ретренинга. Студент на защите должен лично продемонстрировать живое демо: показать, как при симуляции дрейфа данных его код самостоятельно реагирует, запускает переобучение и обновляет модель в Docker.

5. КРИТЕРИИ ОЦЕНИВАНИЯ ВЕХИ 4 (ФИНАЛЬНАЯ ПРИЕМКА MVP)

Финальный продукт оценивается в рамках защиты курсового проекта по следующим критериям Definition of Done (DoD) :

- **9–10 баллов («Отлично» / Уровень Архитектора):**
 - Продукт полностью отчуждаем. Инференс-сервис на FastAPI успешно упакован в Docker-контейнер и запускается одной командой.
 - Все эксперименты, метрики и веса версионизируются внутри MLflow.
 - Реализован сквозной работающий контур продуктивного мониторинга и автоматического ретренинга при фиксации дрейфа данных (concept drift).
 - Репозиторий оформлен эталонно: файл `README.md` содержит подробное руководство по разворачиванию, код написан по PEP-8. Команда блестяще защитила бизнес-эффект перед комиссией.

- **6–8 баллов («Хорошо» / Уровень Разработчика):**
 - Сервис на FastAPI создан, успешно упакован в Docker и выдает корректные прогнозы по эндпоинту /predict.
 - Интегрировано логирование в MLflow, однако в систему записываются только финальные метрики (без сохранения самих артефактов моделей).
 - Контур мониторинга дрейфа и автоматического переобучения отсутствует, либо ретренинг запускается только вручную (автоматический триггер не реализован).
- **4–5 баллов («Удовлетворительно» / Минимальный порог):**
 - **Нижний блокирующий порог.** Студент написал простейший скрипт на FastAPI, который загружает локальную модель и выдает предсказание.
 - Docker-контейнер отсутствует, либо образ собран с критическими ошибками (весит более 5 ГБ, падает при запуске). Инструменты MLflow, Airflow и мониторинг дрейфа проигнорированы полностью. Документация в репозитории носит формальный характер.
- **0–3 балла («Неудовлетворительно»):** Сервис не работает, прогнозы не возвращаются, или на устной защите команда не смогла объяснить архитектуру и синтаксис собственного кода.